

Tabular and Semi-Gradient RL on Text Flappy Bird

Daniel Fernández de la Mela

CentraleSupélec, Université Paris-Saclay
daniel.fernandezdelamela@student-cs.fr

1 Experimental Setup (Q1)

We implement and compare two agents on **TextFlappyBird-v0** (observations: integer $(\Delta x, \Delta y)$ distances to the next pipe gap; actions: $\{0=\text{no-flap}, 1=\text{flap}\}$; reward: +1 per step survived).

Agent 1 — Constant- α First-Visit MC Control. Generates complete episodes under ε -greedy exploration, then updates the Q-table for each first-visited (s, a) pair using the discounted return G_t :

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a)) \quad (1)$$

States are integer tuples used as direct hash keys (no discretization needed).

Agent 2 — True Online Sarsa(λ) [1]. Updates weights at every step via linear approximation $\hat{q}(s, a, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s, a)$ with Dutch eligibility traces (S&B §12.7):

$$\delta = R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w}) \quad (2)$$

$$\mathbf{z} \leftarrow \gamma \lambda \mathbf{z} + (1 - \alpha \gamma \lambda \mathbf{z}^\top \mathbf{x}) \mathbf{x} \quad (3)$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha(\delta + q - q_{\text{old}}) \mathbf{z} - \alpha(q - q_{\text{old}}) \mathbf{x} \quad (4)$$

Features $\mathbf{x}(s, a)$ use tile coding: 8 tilings $\times 8 \times 8$ tiles, IHT size 4096, $\alpha = \alpha_{\text{base}}/8$.

Key differences. Both agents are trained for 50 000 episodes per seed (3 seeds), with ε decaying from 1.0 to 0.05 at rate 0.9999 per episode. Training convergence and eval curves are shown in Fig. 1.

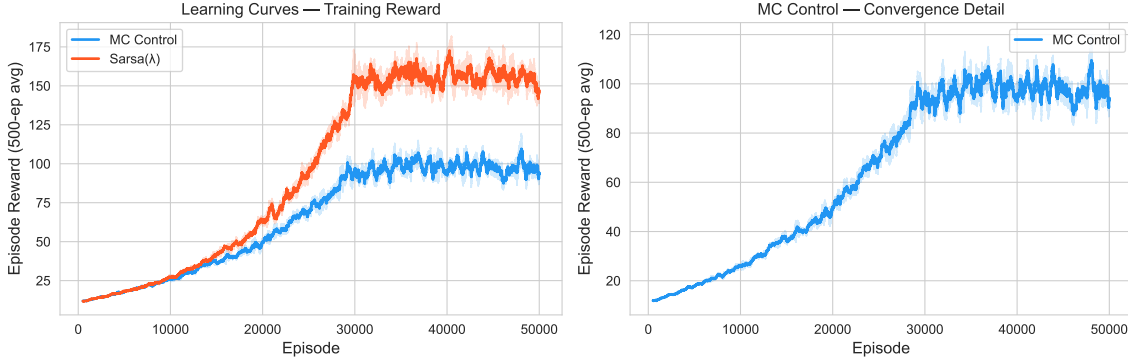


Fig. 1: Learning curves (mean $\pm$ std, 3 seeds, 500-ep smoothing). Left: both agents. Right: MC detail.

MC Control trains in ≈ 60 s/seed (274-state Q-table); Sarsa(λ) requires ≈ 500 s/seed ($8 \times$ slower, 978/4096 non-zero weights). Both plateau around episode 30 000. Under default hyperparameters (MC: $\alpha = 0.1$, $\gamma = 0.99$; Sarsa: $\alpha_{\text{base}} = 0.2$, $\lambda = 0.9$), eval scores over 50 greedy episodes are: MC seeds 0–2: 206 ± 223 , 400 ± 259 , 170 ± 164 ; Sarsa seeds 0–2: 1950 ± 1909 , 603 ± 567 , 370 ± 336 . Sarsa achieves higher but more variable performance — a bimodal distribution typical of Flappy Bird where one suboptimal decision cascades into immediate death.

2 State/Action-Value Functions

Figure 2 shows the learned value function $V(s) = \max_a Q(s, a)$ and greedy policy $\pi(s)$ for both agents after training.

Both agents (Fig. 2) learn the correct policy: flap below the gap ($\Delta y < 0$), do nothing above. MC’s map covers only the 274 visited integer states — coarse but exact. Sarsa’s is smooth: tile coding generalizes across unvisited states, visible in the gradient structure absent from MC. Values peak near $\Delta y = 0$ (gap center) and drop towards boundaries.

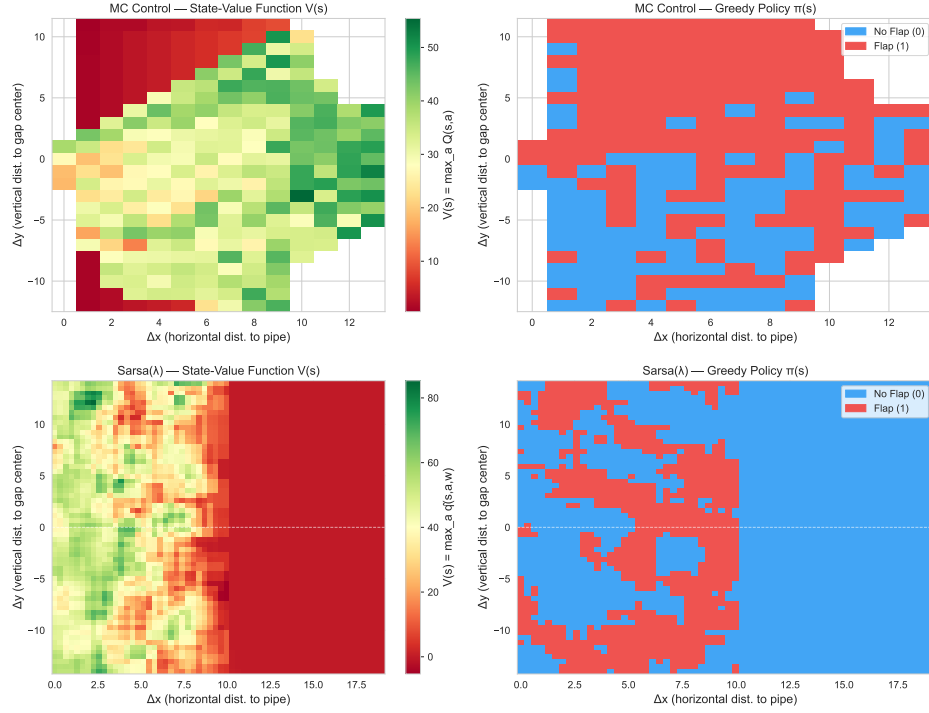


Fig. 2: Value function $V(s)$ and greedy policy. Top: MC (Q-table, 274 states). Bottom: Sarsa(λ) (weight vector). Blue=No Flap, Red=Flap; dashed= $\Delta y=0$.

3 Parameter Sweeps (Q1 cont.)

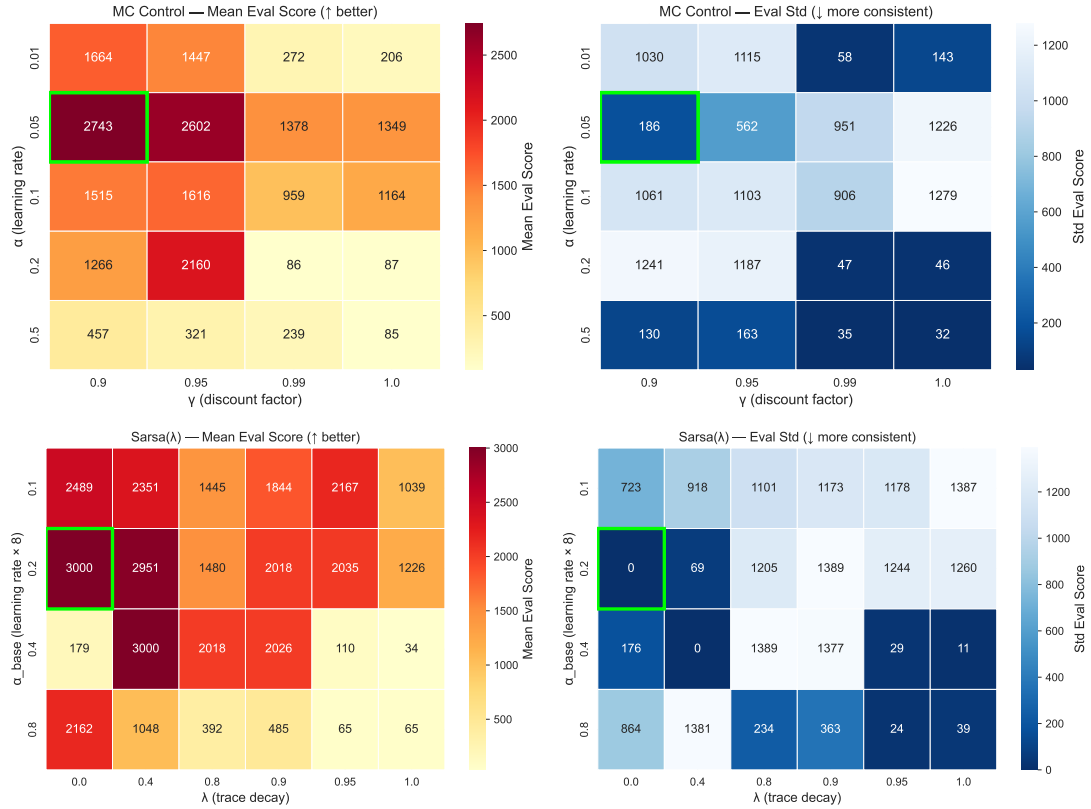


Fig. 3: Parameter sweeps (3 seeds, 30 000 ep.). **Top:** MC Control ($\alpha \times \gamma$); green = best ($\alpha = 0.05, \gamma = 0.9$, mean 2743). **Bottom:** Sarsa(λ) ($\alpha_{\text{base}} \times \lambda$); green = best; 3000 = eval cap (perfect).

MC Control (Fig. 3, top): Best config $\alpha = 0.05$, $\gamma = 0.9$ (mean eval 2743, std 186). $\gamma = 0.9$ outperforms $\gamma = 0.99$: the effective horizon $1/(1 - \gamma) \approx 10$ steps matches the decision scale of TFB (one pipe). Higher γ over-weights distant, hard-to-control events.

Sarsa(λ) (Fig. 3, bottom): Two configs reach the cap (3000/3000 with zero variance): ($\alpha_{\text{base}} = 0.2$, $\lambda = 0.0$) and ($\alpha_{\text{base}} = 0.4$, $\lambda = 0.4$). Configs with $\lambda \geq 0.95$ collapse: long traces propagate credit beyond the 1–2 step consequence window of each flap. $\lambda = 0$ (TD(0)) is optimal — credit assignment is exact for this short-horizon task.

4 Additional Experiments

4.1 TFB-v0 vs screen-v0 (Q2)

TextFlappyBird-v0 exposes $(\Delta x, \Delta y)$ — a minimal sufficient statistic — enabling tabular MC with only 274 states. TextFlappyBird-screen-v0 provides the full rendered frame (hashed to tuples for MC): the Q-table grows to 10075 states ($37\times$ larger), yet MC scores 1815 ± 1051 , below the cap. The screen encodes irrelevant layout features (pipe walls, borders) that inflate the state space without adding decision-relevant information. Sarsa(λ) tile coding is inapplicable to screen observations without a compact embedding.

4.2 Original Flappy Bird (Q3)

flappy-bird-gym [3] returns RGB frames ($512 \times 288 \times 3$). MC is impractical (state space never repeats); Sarsa tile coding assumes low-dimensional inputs. Both agents *could* work given a hand-crafted feature extractor (e.g., extracting pipe distances from pixels), but are inapplicable out-of-the-box. Deep RL (DQN, PPO) is the appropriate choice for raw pixel environments.

4.3 Generalization to Different Configurations (Q4)

Table 1: Generalization: trained on default (H=15, W=20, g=4), tested zero-shot (seed 42, 100 ep.).

Configuration	MC eval	Sarsa eval
Baseline ($g = 4$, same)	240 ± 205	202 ± 200
Harder ($g = 3$)	21 ± 18	22 ± 14
Easier ($g = 6$)	747 ± 686	645 ± 589
Bigger screen (20×30)	4 ± 0	4 ± 0
Smaller screen (10×15)	105 ± 104	62 ± 63

Results (Table 1) show performance scales with gap size but fails completely on unseen screen sizes. The 20×30 grid scores 4 ± 0 for both — immediate death: MC has no Q-values for the new integer states; Sarsa’s tile boundaries map to out-of-distribution features. The smaller screen partially transfers for MC (105) but less for Sarsa (62). Both agents learn environment-specific rather than geometry-invariant policies.

References

1. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press (2018)
2. Christodoulidis, S.: Text Flappy Bird Gym. <https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym> (2022)
3. Talendar: Flappy Bird Gym. <https://github.com/Talendar/flappy-bird-gym> (2021)